

Deep Reinforcement Learning Algorithms

Deep Q-Learning

Approximate $Q(x, u)$ using a neural network: $\hat{Q}(x, u; \theta)$, where θ is the weights of a neural network.

Deep Q-Learning

Approximate $Q(x, u)$ using a neural network: $\hat{Q}(x, u; \theta)$, where θ is the weights of a neural network.

Learning objective

$$\min_{\theta} E \left[\frac{1}{2} \left(r(x_k, u_k) + \alpha \max_v \hat{Q}(x_{k+1}, v; \theta) - \hat{Q}(x_k, u_k; \theta) \right)^2 \right]$$

Deep Q-Learning

Approximate $Q(x, u)$ using a neural network: $\hat{Q}(x, u; \theta)$, where θ is the weights of a neural network.

Learning objective

$$\min_{\theta} E \left[\frac{1}{2} \left(r(x_k, u_k) + \alpha \max_v \hat{Q}(x_{k+1}, v; \theta) - \hat{Q}(x_k, u_k; \theta) \right)^2 \right]$$

Q-learning with neural networks

$$\theta \leftarrow \theta + \beta_k \delta_k \nabla_{\theta} \hat{Q}(x_k, u_k; \theta),$$

where

$$\delta_k = r(x_k, u_k) + \alpha \max_v \hat{Q}(x_{k+1}, v; \theta) - \hat{Q}(x_k, u_k; \theta)$$

Deep Q-Learning

Issues of deep reinforcement learning

- No theoretical guarantees on optimality and convergence
- More unstable during training

Replay Buffer and Experience Replay

Pretend we have a supervised learning problem

$$\frac{1}{2} \min_{\theta} \sum_k \left(y_k - \hat{Q}(x_k, u_k; \theta) \right)^2,$$

where $y_k = r(x_k, u_k) + \alpha \max_v \hat{Q}(x_{k+1}, v; \theta)$ (we ignore the fact that y_k is a function of θ like in linear function approximation.)

Replay Buffer and Experience Replay

Pretend we have a supervised learning problem

$$\frac{1}{2} \min_{\theta} \sum_k \left(y_k - \hat{Q}(x_k, u_k; \theta) \right)^2,$$

where $y_k = r(x_k, u_k) + \alpha \max_v \hat{Q}(x_{k+1}, v; \theta)$ (we ignore the fact that y_k is a function of θ like in linear function approximation.)

Stochastic Gradient Descent (SGD)

$$\theta \leftarrow \theta + \beta_k (y_k - \hat{Q}(x_k, u_k; \theta)) \nabla_{\theta} \hat{Q}(x_k, u_k; \theta)$$

with *independently and identically distributed* data samples $(y_k, (x_k, u_k))$.

Replay Buffer and Experience Replay

Stochastic Gradient Descent (SGD)

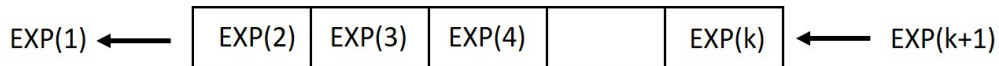
$$\theta \leftarrow \theta + \beta_k (y_k - \hat{Q}(x_k, u_k; \theta)) \nabla_{\theta} \hat{Q}(x_k, u_k; \theta)$$

with *independently and identically distributed* data samples $(y_k, (x_k, u_k))$.

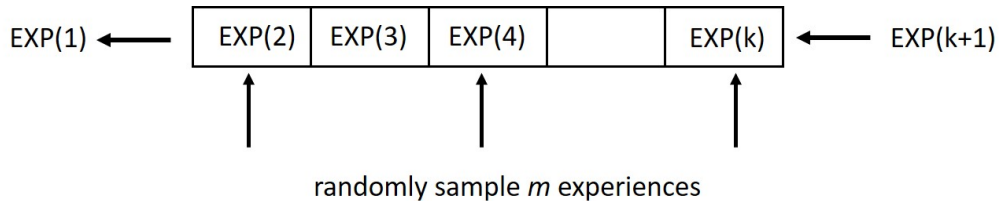
$$(y_1, (x_1, u_1)) \rightarrow (y_2, (x_2, u_2)) \rightarrow (y_3, (x_3, u_3)) \rightarrow \dots$$

are highly correlated samples in reinforcement learning applications.

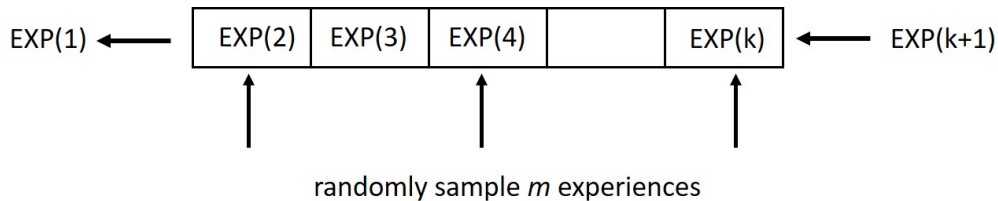
Replay Buffer



Replay Buffer and Experience Replay



Replay Buffer and Experience Replay



Minibatch SGD

$$\theta \leftarrow \theta + \beta_k \sum_{b=1}^m (y_{k,b} - \hat{Q}(x_{k,b}, u_{k,b}; \theta)) \nabla_{\theta} \hat{Q}(x_{k,b}, u_{k,b}; \theta).$$

Target Network

From the supervised learning perspective, the target value $y_{k,m} = r(x_{k,m}, u_{k,m}) + \alpha \max_v \hat{Q}(x_{k+1,m}, v; \theta)$ should be independent of θ .

Target Network

From the supervised learning perspective, the target value $y_{k,m} = r(x_{k,m}, u_{k,m}) + \alpha \max_v \hat{Q}(x_{k+1,m}, v; \theta)$ should be independent of θ .

Target Network Q_{target}

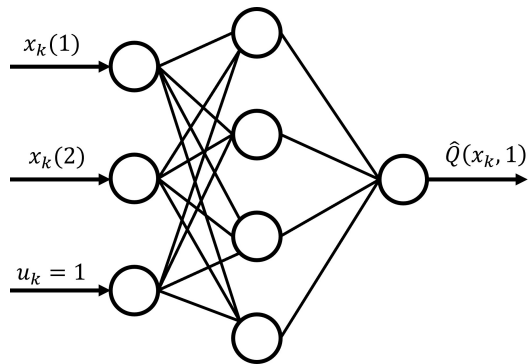
copy θ to θ_{target} once every n iterations and use Q_{target} to calculate $y_{k,m}$.

Deep Q-Network (DQN)

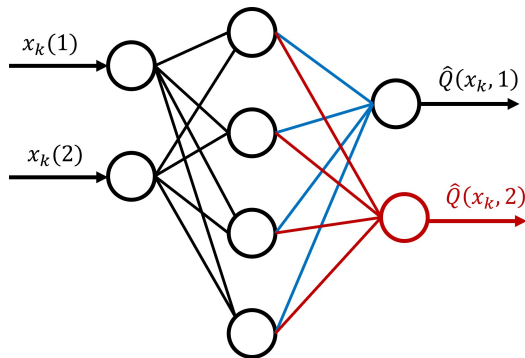
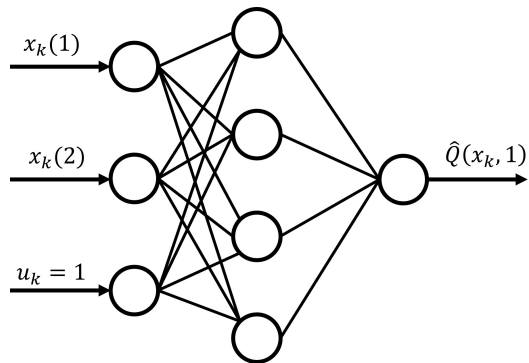
How to calculate $\sum_{b=1}^m (y_{k,b} - \hat{Q}(x_{k,b}, u_{k,b}; \theta)) \nabla_{\theta} \hat{Q}(x_{k,b}, u_{k,b}; \theta)$ and update θ ?

Deep Q-Learning in PyTorch

Deep Q-Network Architecture



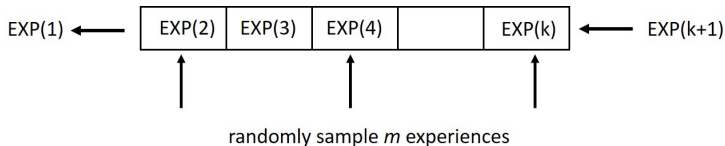
Deep Q-Network Architecture



Couple the estimation of Q-values ($Q(x, u)$) of different actions u for the same state x .

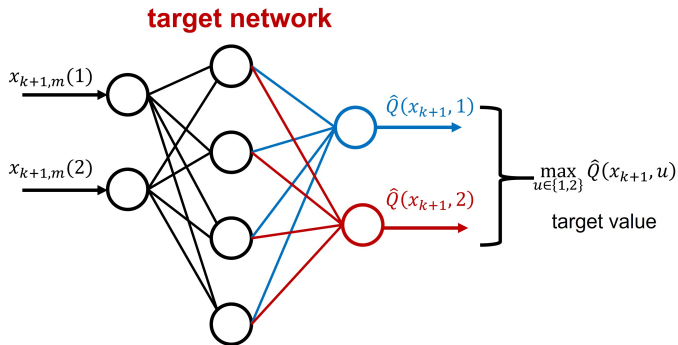
Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

- Sample a mini-batch $\{(x_{k,b}, u_{k,b}, r_{k,b}, x_{k+1,b})\}_{b=1,\dots,m}$.
`random.sample(self.replay_buffer, batch_size)`



Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

- Generate target-values $y_{k,m}$ using the target network
`next_state_values = target_net(next_states).max(1)[0]`
`target_values = rewards + gamma × next_state_values.detach()`

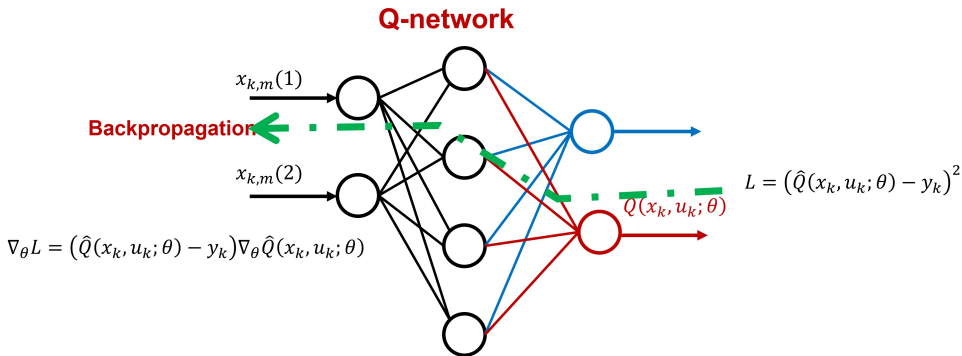


Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

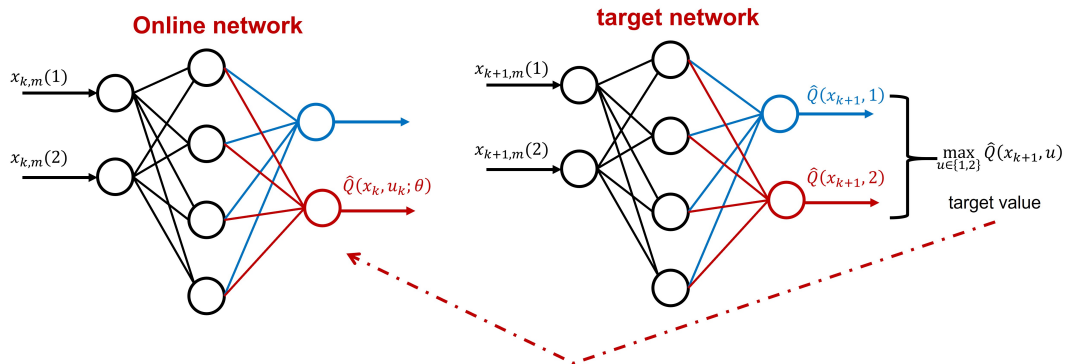
- Define a loss function $L(\theta) = \sum_{b=1}^m \left(\hat{Q}(x_{k,b}, u_{k,b}; \theta) - y_{k,b} \right)^2$.
`loss = nn.MSELoss()(state_action_values, target_values)`

Deep-Q-Network (DQN) (Minh et al 2015) in PyTorch

- Update $\theta \leftarrow \theta - \beta_k \nabla_{\theta} L(\theta)$
`optimizer.zero_grad()` `loss.backward()` `optimizer.step()`

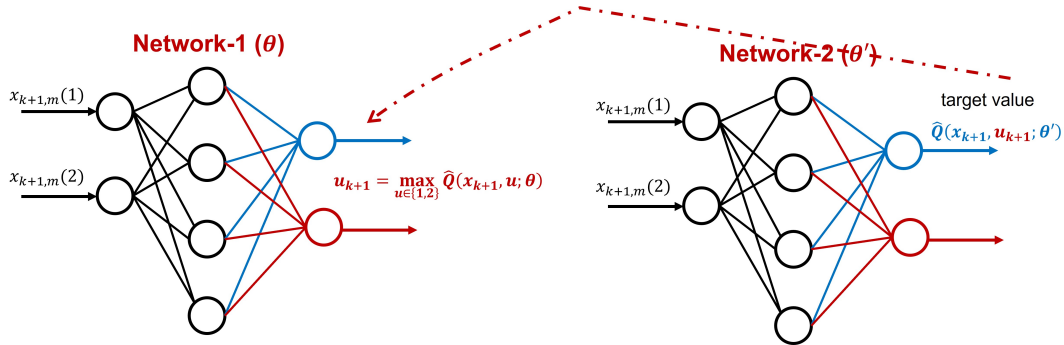


Double DQN (van Hasselt, Guez, Silver 2015)



Target network is used to (i) select the action u (ii) evaluate $Q(x_{k+1}, u)$. Selecting the optimal action and evaluating the Q-function together lead to an overestimation.

Double DQN (van Hasselt, Guez, Silver 2015)



Double DQN

Separate action selection and evaluation. Both Q-networks are trained at each iteration by using the other one as the target-network.

Clipped Double-Q Learning (Fujimoto, van Hoof, David Meger 2018)

Maintain two Q-networks, denoted by their parameters θ and θ' ,

$$\theta \leftarrow \theta + \beta_k \sum_b (y_{k,b} - Q(x_{k,b}, u_{k,b}; \theta)) \nabla_{\theta} Q(x_{k,b}, u_{k,b}; \theta)$$

$$\theta' \leftarrow \theta' + \beta_k \sum_b (y_{k,b} - Q(x_{k,b}, u_{k,b}; \theta')) \nabla_{\theta'} Q(x_{k,b}, u_{k,b}; \theta')$$

where

$$y_{k,b} = r_{k,b} + \alpha \text{min} \left\{ \max_u Q(x_{k+1,b}, u; \theta), \max_u Q(x_{k+1,b}, u; \theta') \right\}$$

Policy Gradient and Actor-Critic Algorithms

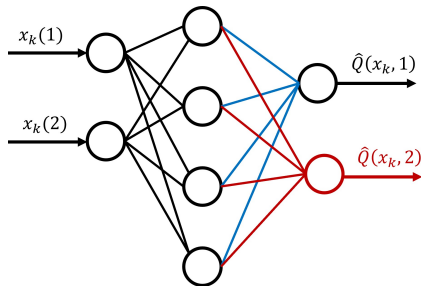
Q-learning follows value iteration

$$\tilde{Q}(i, u) \leftarrow \tilde{Q}(i, u) + \beta_k \left(r(i, u) + \alpha \max_v \tilde{Q}(j, v) - \tilde{Q}(i, u) \right)$$

Q-learning follows value iteration

$$\tilde{Q}(i, u) \leftarrow \tilde{Q}(i, u) + \beta_k \left(r(i, u) + \alpha \max_v \tilde{Q}(j, v) - \tilde{Q}(i, u) \right)$$

Works well when the action space is finite.



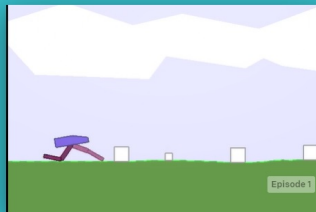
Policy Iteration

How about applications with infinite (countable or uncountable) action spaces?

BipedalWalkerHardcore-v2

Hardcore version with ladders, stumps, pitfalls. Time limit is increased due to obstacles. Reward is given for moving forward, total 300+ points up to the far end. If the robot falls, it gets -100. Applying motor torque costs a small amount of points, more optimal agent will get better score. State consists of hull angle speed, angular velocity, horizontal speed, vertical speed, position of joints and joints angular speed, legs contact with ground, and 10 lidar rangefinder measurements. There's no coordinates in the state vector.

[VIEW SOURCE ON GITHUB](#)



RandomAgent on BipedalWalkerHardcore-v2

Policy Iteration

How about applications with infinite (countable or uncountable) action spaces?

Policy Iteration

Given policy μ_k ,

- policy evaluation to compute V_{μ_k}

$$V_{\mu_k}(i) = E[r(i, u)] + \alpha \sum_j P_{ij}(u) V_{\mu_k}(j)$$

- policy improvement to obtain a new policy μ_{k+1}

Deep Actor-Critic Algorithms

Actor-critic: generalized policy-iteration

- Critic: estimate the value of the current policy (Q-function in general, because the actor needs the action values)
TD(λ), double-Q, clipped double-Q, etc.
- Actor:
 - ▶ ϵ -greedy based on the current Q-function
 - ▶ Policy-gradient

Actor-Critic

Parameterized Policy

$\pi_{\theta}(u|x)$: probability of choosing action u in state x . The policy (probability) is parameterized by θ .

Actor-Critic

Parameterized Policy

$\pi_{\theta}(u|x)$: probability of choosing action u in state x . The policy (probability) is parameterized by θ .

Given Q-function (action value function),

$$V_{\theta^*} = \max_{\theta} E \left[\sum_u Q(x, u) \pi_{\theta}(u|x) \right].$$

Examples

Finite action space: Gibbs policy

$$\pi_{\theta}(a|x) = \frac{\exp(\theta^T \phi(x, a))}{\sum_{u \in A} \exp(\theta^T \phi(x, u))}$$

where $\phi(x, u)$ is the feature vector.

Examples

Finite action space: Gibbs policy

$$\pi_{\theta}(a|x) = \frac{\exp(\theta^T \phi(x, a))}{\sum_{u \in A} \exp(\theta^T \phi(x, u))}$$

where $\phi(x, u)$ is the feature vector.

Continuous action space: Gaussian policy

$$\pi_{\theta}(a|x) = \frac{1}{\sqrt{(2\pi)^d \det(\Sigma_{\theta}(x))}} \exp \{ -(a - \mu_{\theta}(x))^T \Sigma_{\theta}^{-1}(x) (a - \mu_{\theta}(x)) \}$$

where $\mu_{\theta}(x) = \phi^T(x)\theta$ and $\Sigma_{\theta}(x) = \sigma^2 I$.

Policy Gradient

$$\theta \leftarrow \theta + \beta \nabla_{\theta} V_{\theta}$$

Policy Gradient Theorem

$$\nabla_{\theta} V_{\theta} = E_{x \sim \rho_{\theta}, a \sim \pi_{\theta}(a|x)} [Q^{\pi_{\theta}}(x, a) \nabla_{\theta} \log \pi_{\theta}(a|x)],$$

where $\rho_0(x)$ is the initial distribution of the states and

$$\rho_{\theta}(x) = \sum_{k=0}^{\infty} \alpha^k \Pr(x_k = x),$$

called (improper) discounted state distribution.

Actor-critic

$\nabla_{\theta} \log \pi_{\theta}(a|x)$: score function

- Gibbs policy: $\nabla_{\theta} \log \pi_{\theta}(a|x) = \phi(x, a) - E_{\pi_{\theta}}[\phi(x, \cdot)]$
- Gaussian policy: $\nabla_{\theta} \log \pi_{\theta}(a|x) = \frac{(a - \mu(x))\phi(x)}{\sigma^2}$ where $\mu(x) = \phi^T(x)\theta$

REINFORCE (Williams (1988, 1992))

Generate an episode $x_0, a_0, x_1, a_1, \dots, x_{T-1}, a_{T-1}, x_T$ by following policy π_θ . For each step t of the episode, update θ as follows:

$$\theta \leftarrow \theta + \beta \alpha^t \nabla_\theta \log \pi_\theta(a_t | x_t) \sum_{\tau=t}^T \alpha^{\tau-t} r_\tau.$$

Continue to the next episode with the updated θ .

Monte-Carlo Policy

REINFORCE uses the Monte-Carlo method

$$Q_\theta(x_t, a_t) \approx \sum_{\tau=t}^T \alpha^{\tau-t} r_\tau.$$

Variance Reduction and Deep Policy-Gradient Algorithms

Variance Reduction (Control Variates Method)

We are interested in computing

$$E[f(x)] \approx \underbrace{\frac{1}{N} \sum_{i=1}^N f(x_i)}_F \quad x_i \sim P(x)$$

But F (finite-sample average) may have a high variance.

Variance Reduction

Replace F with F' such that

$$E[F] = E[F'], \text{ Var}(F') \leq \text{Var}(F)$$

Variance Reduction

Consider function $\phi(x)$ such that $E[\phi(x)] = 0$,

$$E[F(x) - \phi(x)] = E[F(x)]$$

$$\text{Var}(F(x) - \phi(x)) = \text{Var}(F(x)) - 2\text{Cov}(F(x), \phi(x)) + \text{Var}(\phi(x))$$

The variance can be reduced when $\phi(x)$ is strongly correlated with $F(x)$.

Estimate $\nabla_{\theta} V(i)$

$$\nabla_{\theta} V(i) = \sum_{k=0}^T \alpha^k \nabla \log \pi_{\theta}(u_k | x_k) \times \underbrace{(r(x_k, u_k) + \alpha V_{\theta}(x_{k+1}) - V_{\theta}(x_k))}_{\text{TD error}}.$$

Actor-Critic

- Advantage Actor-Critic:

$$A(s, a) = Q_{\theta}(s, a) - V_{\tilde{\theta}}(s)$$

- TD Actor-Critic:

$$A(s, a) = r(s, a) + \alpha V_{\theta}(s') - V_{\theta}(s)$$

TD error estimates the advantage function.

Actor-Critic with Neural Networks

Critic

double-Q, target-Q, and clipped-Q

Actor-Critic with Neural Networks

Critic

double-Q, target-Q, and clipped-Q

Actor

Sample a mini-batch sampled from replay buffer. Train the policy network with a weighted cross-entropy loss:

$$L = - \sum_{(s,a)} A(s, a) \log \pi_{\theta}(a|s)$$

where

$$A(s, a) = r(s, a) + \alpha V_{\theta}(s') - V_{\theta}(s).$$

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

Critic

Q-network $Q_w(s, a)$: trained with mini-batch and temporal-difference.

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

Critic

Q-network $Q_w(s, a)$: trained with mini-batch and temporal-difference.

Actor

Deterministic policy-network $\mu_\theta(x)$: trained with loss function

$$L(\theta) = - \sum_{s_b \in \text{minibatch}} Q_w(s_b, \mu_\theta(s_b)).$$

Deep deterministic policy gradient (DDPG) (Lillicrap et al. 2016)

Critic

Q-network $Q_w(s, a)$: trained with mini-batch and temporal-difference.

Actor

Deterministic policy-network $\mu_\theta(x)$: trained with loss function

$$L(\theta) = - \sum_{s_b \in \text{minibatch}} Q_w(s_b, \mu_\theta(s_b)).$$

Twin Delayed DDPG (TD3) (Fujimoto, van Hoof, Meger, 2018): Clipped double-Q + deterministic PG.